

Statistical Pattern Recognition

Bayesian Gaussian Classification and Minimum-Risk Decision Making

Technical Code and Methodology Report

This report analyzes the supplied corrected Python implementation for a statistical pattern-recognition assignment. It covers the software structure, probabilistic model, Bayes Minimum Error (BME) and Bayes Minimum Risk (BMR) decision rules, maximum-likelihood Gaussian estimation, leave-one-out evaluation, synthetic data generation, artifact handling, and reproducibility. Figures and experimental-result visualizations are intentionally omitted.

Contents

Scope and Source-Fidelity Note	3
1 Abstract	3
2 Introduction	3
3 Project Overview	4
4 Technical Foundations	5
4.1 Bayesian Classification	5
4.2 Multivariate Gaussian Class Model	5
4.3 Log-Density Formulation	5
4.4 Bayes Minimum Error	6
4.5 Bayes Minimum Risk	6
5 Data and Input Processing	6
5.1 Exercise 1: Synthetic Two-Class Gaussian Data	6
5.2 Exercise 2a: Iris	6
5.3 Exercise 2b: Liquid-equivalent Dataset	7
5.4 Exercise 2c: Normal Train/Test Data	7
6 System Architecture and Code Organization	7
6.1 Configuration and Environment Capture	7
6.2 Low-Level Numerical Primitive	7
6.3 Posterior Normalization	8
6.4 Binary Decision Functions	8
6.5 Multiclass Generalization	8
7 Detailed Methodology	8
7.1 Exercise 1 Simulation Pipeline	8
7.2 Maximum-Likelihood Gaussian Parameter Estimation	9
7.3 True Leave-One-Out Evaluation	9
7.4 Train/Test Evaluation for the Normal Dataset	9
8 Mathematical Formulation	10
8.1 Binary Asymmetric-Risk Boundary	10
8.2 Error Metrics	10
9 Implementation Details	10
9.1 Vectorization and Array Semantics	10
9.2 Positive-Definiteness Checks	10
9.3 Artifact Naming and Data Lineage	11
9.4 Source-Level Traceability	11
10 Execution Workflow	12
11 Design Decisions and Rationale	12
11.1 Why Log Probabilities Are Used	12

11.2 Why Maximum-Likelihood Covariance Uses n	12
11.3 Why Leave-One-Out Is Implemented Manually	12
11.4 Why the Missing Data Are Reconstructed or Substituted	12
12 Computational Considerations	12
13 Reproducibility and Reliability	13
14 Limitations and Technical Considerations	13
14.1 Missing Assignment Data	13
14.2 Repeated Matrix Inversion	13
14.3 Conditioning and Covariance Regularization	14
14.4 Equal Priors	14
14.5 Plotting Code and Reporting Scope	14
15 Overall Technical Assessment	14
16 Conclusion	14

Scope and Source-Fidelity Note

The analysis is based on the complete supplied Python source file. The implementation itself documents an important assignment-to-source discrepancy: its comments state that the assignment sheet labels Exercise 1 as Computer Exercise 2.5, while the parameter values and loss matrix encoded in the original submission correspond to a different textbook exercise. The corrected program intentionally preserves the mathematical problem represented by the original code instead of silently replacing it.

The source also states that two Normal-data files and the assignment-specific Liquid data file were not supplied. The implementation therefore generates the two-dimensional Normal training and testing samples from the parameters given in the assignment and uses the standard scikit-learn Wine dataset, restricted to six original chemical measurements, as a documented substitute for the missing Liquid database. These choices describe the implementation; they should not be mistaken for the unavailable original data.

The source contains plotting routines and saves plot images, but this report omits those figures because it focuses on the code and methodology rather than on results or visualization.

1. Abstract

The supplied implementation is a structured statistical pattern-recognition program centered on generative Gaussian classification. Its core abstraction is a reusable multivariate Gaussian log-density routine that feeds both Bayes Minimum Error and Bayes Minimum Risk decision rules. The same framework is then extended to a multiclass setting by estimating class-specific Gaussian means and covariance matrices with maximum likelihood. The resulting classifier is evaluated with different validation designs: direct simulation for a two-class textbook exercise, 150-fold and 178-fold leave-one-out classification for two multiclass databases, and a fixed train/test protocol for a two-class Normal dataset.

From a software-engineering perspective, the implementation separates model-independent utilities from experiment-specific orchestration. It explicitly records priors, covariance structures, loss matrices, sample counts, random seeds, package versions, configuration parameters, and generated artifacts. It also writes structured CSV files and a JSON reproducibility manifest. A central design choice is to perform the probability calculations in the logarithmic domain, which reduces the risk of numerical underflow compared with direct Gaussian-density evaluation.

Two implementation details are important when interpreting the program. The code performs the intended Gaussian calculations consistently, but it recomputes covariance inverses and log-determinants inside the low-level density routine rather than caching them, which is computationally unnecessary when the same covariance is reused. In addition, the missing-data substitutions mean that the resulting Exercise 2b and Exercise 2c outputs are not equivalent to results produced from the unavailable original files. These are characteristics of the implementation and its execution context, not theoretical flaws in Gaussian Bayes classification.

2. Introduction

Statistical pattern recognition formulates classification as a decision problem under uncertainty. Rather than mapping a feature vector directly to a class label through a purely discriminative rule, a generative classifier models the conditional distribution of the features given each class and

combines those class-conditional models with prior probabilities. The supplied code implements exactly this viewpoint for Gaussian class models.

The project has two closely related layers. The first is a controlled two-class experiment in which the class-conditional distributions are known analytically. This part compares two Bayesian decision criteria: minimizing expected classification error and minimizing expected loss under an explicit asymmetric loss matrix. The second layer turns the same Bayesian machinery into a data-driven multiclass classifier by estimating Gaussian means and covariance matrices from training data using maximum likelihood.

The implementation therefore serves as an executable demonstration of several fundamental ideas in statistical pattern recognition: multivariate Gaussian modeling, likelihood and log-likelihood computation, posterior inference, Bayes decision theory, asymmetric risk, maximum-likelihood parameter estimation, leave-one-out cross-validation, train/test evaluation, confusion-matrix construction, and reproducibility-oriented artifact generation.

The report focuses on the implementation itself. It does not infer experimental conclusions beyond what is structurally supported by the code, and it does not reproduce result plots or numerical outcome tables.

3. Project Overview

The software can be understood as a pipeline with four conceptual layers.

1. **Configuration and environment capture.** The program imports numerical, data-handling, visualization, and statistical-learning libraries, establishes a deterministic random seed, creates an output directory, and records interpreter and package versions.
2. **Bayesian Gaussian decision primitives.** Low-level functions compute multivariate Gaussian log densities, posterior probabilities, BME decisions, BMR decisions, confusion matrices, and error statistics.
3. **Experiment drivers.** The first experiment generates Gaussian samples directly from specified class distributions. The second experiment estimates Gaussian parameters from observed data and then performs multiclass BME classification under leave-one-out or train/test protocols.
4. **Artifact and reproducibility layer.** Each experiment writes structured CSV files containing generated data, parameter summaries, misclassification details, performance summaries, and a final JSON manifest.

The implementation is not just a sequence of disconnected steps. Its functions form a reusable classifier toolkit, which is then applied to the assignment scenarios in the later parts of the program.

Table 1: High-level mapping between source components and responsibilities.

Source component	Primary responsibility
Configuration block	Imports, random seed, output directory, runtime metadata.
<code>log_gaussian_pdf</code>	Multivariate Gaussian log-density evaluation.
<code>posterior_probabilities</code>	Stable conversion of class scores into normalized posteriors.
<code>bme_predict</code>	Minimum-error Bayesian decision rule.
<code>bmr_predict</code>	Minimum-risk decision rule using the loss matrix.
<code>fit_gaussian_class_parameters</code>	Estimation of class means and covariance matrices.
<code>multiclass_bme_predict</code>	Multiclass Gaussian BME classification.
<code>leave_one_out_bme</code>	Refit-and-predict loop for true leave-one-out evaluation.
Experiment blocks	Construction of Exercise 1, Iris, Wine-equivalent Liquid, and Normal datasets.
Artifact layer	CSV summaries, detailed records, and JSON manifest generation.

4. Technical Foundations

4.1 Bayesian Classification

Let $\mathbf{x} \in \mathbb{R}^d$ denote a feature vector and let C_i denote class i . A generative classifier starts from the class-conditional density $p(\mathbf{x} \mid C_i)$ and a prior probability $P(C_i)$. Bayes' rule gives

$$P(C_i \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid C_i)P(C_i)}{\sum_{j=1}^K p(\mathbf{x} \mid C_j)P(C_j)}. \quad (1)$$

The supplied code uses this relationship in two ways. For BME classification it is sufficient to compare the unnormalized joint terms $p(\mathbf{x} \mid C_i)P(C_i)$. For BMR classification it additionally normalizes those joint terms to obtain posterior probabilities because the loss function operates on expected conditional risk.

4.2 Multivariate Gaussian Class Model

For a d -dimensional Gaussian distribution with mean vector $\boldsymbol{\mu}_i$ and covariance matrix $\boldsymbol{\Sigma}_i$, the class-conditional density is

$$p(\mathbf{x} \mid C_i) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_i|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1} (\mathbf{x} - \boldsymbol{\mu}_i) \right]. \quad (2)$$

The quadratic form measures the squared Mahalanobis distance from the class mean. In the implementation, this expression is evaluated in logarithmic form.

4.3 Log-Density Formulation

Taking the logarithm produces

$$\log p(\mathbf{x} \mid C_i) = -\frac{1}{2} \left[d \log(2\pi) + \log |\boldsymbol{\Sigma}_i| + (\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1} (\mathbf{x} - \boldsymbol{\mu}_i) \right]. \quad (3)$$

This transformation is mathematically equivalent to evaluating the density directly, but it is numerically safer because very small Gaussian densities can underflow in floating-point arithmetic. The code computes the log-determinant with `np.linalg.slogdet` and evaluates the quadratic form with `np.einsum`.

4.4 Bayes Minimum Error

Under zero-one loss, the Bayes Minimum Error decision is

$$\hat{c}_{\text{BME}}(\mathbf{x}) = \underset{i}{\operatorname{argmax}} \{ \log p(\mathbf{x} \mid C_i) + \log P(C_i) \}. \quad (4)$$

The implementation performs precisely this `argmax` in `bme_predict`. Because the logarithm is monotonic, maximizing the log-joint quantity is equivalent to maximizing the original joint probability.

4.5 Bayes Minimum Risk

For a loss matrix $\mathbf{\Lambda}$ whose entry λ_{ij} is the loss associated with deciding class i when the true class is j , the conditional risk of decision a_i is

$$\mathcal{R}(a_i \mid \mathbf{x}) = \sum_{j=1}^K \lambda_{ij} P(C_j \mid \mathbf{x}). \quad (5)$$

The Bayes decision minimizes this quantity:

$$\hat{c}_{\text{BMR}}(\mathbf{x}) = \underset{i}{\operatorname{argmin}} \mathcal{R}(a_i \mid \mathbf{x}). \quad (6)$$

The Exercise 1 implementation uses

$$\mathbf{\Lambda} = \begin{bmatrix} 0 & 1 \\ 0.5 & 0 \end{bmatrix}. \quad (7)$$

Consequently,

$$\mathcal{R}(a_1 \mid \mathbf{x}) = P(C_2 \mid \mathbf{x}), \quad (8)$$

$$\mathcal{R}(a_2 \mid \mathbf{x}) = 0.5 P(C_1 \mid \mathbf{x}). \quad (9)$$

Thus the code implements an asymmetric decision rule: an error caused by choosing class 1 for a class-2 sample is assigned twice the loss of choosing class 2 for a class-1 sample. This asymmetry can make the BMR boundary differ from the BME boundary even when the underlying class-conditional densities are unchanged.

5. Data and Input Processing

5.1 Exercise 1: Synthetic Two-Class Gaussian Data

The first experiment uses two-dimensional Gaussian classes with equal prior probabilities, a common covariance of $0.2\mathbf{I}_2$, and 100 simulated samples per class. The first class has mean $[1, 1]^\top$. Two scenarios are evaluated for the second mean: $[1.5, 1.5]^\top$ and $[3, 3]^\top$. The samples are generated using `rng.multivariate_normal`, so the implementation corresponds to the stated Gaussian model rather than to a generic uniform random sampling procedure.

5.2 Exercise 2a: Iris

The code loads the standard Iris dataset with 150 observations, three classes, and four features. The feature columns are reordered to match the assignment wording: Petal Length, Sepal Length,

Sepal Width, and Petal Width. Class labels are converted from zero-based dataset labels to one-based labels used throughout the assignment-oriented code.

5.3 Exercise 2b: Liquid-equivalent Dataset

The original Liquid data file was unavailable to the implementation. The code therefore loads the standard scikit-learn Wine dataset, verifies the required class counts of 59, 71, and 48, and retains six original chemical-analysis features: Alcohol, Malic Acid, Ash, Alcalinity of Ash, Magnesium, and Total Phenols. This is a documented substitution, not an implicit claim that the Wine data are identical to the missing Liquid file.

5.4 Exercise 2c: Normal Train/Test Data

The assignment-specific Normal files were also unavailable. The implementation reconstructs deterministic training and testing files from the printed model parameters: two classes with means $[0, 0]^T$ and $[4, 0]^T$, common covariance

$$\Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \quad (10)$$

and 500 observations from each class in each file. Separate seeded random draws are used for training and testing.

6. System Architecture and Code Organization

6.1 Configuration and Environment Capture

The code begins with standard-library modules for paths, JSON serialization, platform information, and runtime metadata. NumPy provides vectorized numerical operations; pandas manages tabular artifacts; Matplotlib provides optional plotting; SciPy supplies Gaussian and normal-distribution functionality; scikit-learn supplies the Iris and Wine datasets.

A module-level integer seed is used to initialize reproducible random-number generators. The output directory is created relative to the current working directory, and the program prints the Python and package versions. This metadata is later duplicated in the final JSON manifest.

6.2 Low-Level Numerical Primitive

The function `log_gaussian_pdf` is the numerical foundation of the entire classifier. It converts its inputs to floating-point arrays, computes the sign and logarithm of the covariance determinant, rejects non-positive-definite covariance matrices, computes the inverse covariance, forms the centered observations, evaluates the Mahalanobis quadratic term, and returns the vector of log densities.

The use of `np.einsum("...i,ij,...j->...", diff, inv_cov, diff)` is noteworthy. The expression implements the quadratic form for one or more observations without an explicit Python loop over samples, which keeps the operation vectorized while remaining general over leading dimensions.

6.3 Posterior Normalization

The `posterior_probabilities` function first creates log-joint scores,

$$\ell_i(\mathbf{x}) = \log p(\mathbf{x} \mid C_i) + \log P(C_i), \quad (11)$$

and then subtracts the maximum score before exponentiating. For a row of scores $\ell_1, \dots, \ell_K$, the implementation effectively computes

$$P(C_i \mid \mathbf{x}) = \frac{\exp(\ell_i - m)}{\sum_{j=1}^K \exp(\ell_j - m)}, \quad m = \max_j \ell_j. \quad (12)$$

The subtraction of m does not change the normalized probabilities; it only improves numerical stability.

6.4 Binary Decision Functions

The binary BME function builds a two-column matrix of log-likelihoods, adds the log priors, and selects the class with the largest score. The BMR function reuses BME, reconstructs the posterior probabilities, multiplies them by the transposed loss matrix, and selects the class with the smallest risk. Reusing the same score calculation keeps the BMR rule consistent with the BME implementation.

One implementation detail deserves mention. The BMR function subtracts the log priors from the log-joint discriminants before passing them to a function that adds the priors again. Algebraically, this recovers the original likelihood scores and preserves the intended posterior calculation.

6.5 Multiclass Generalization

The function `multiclass_bme_predict` generalizes the binary BME rule to an arbitrary class set. Parameter dictionaries are keyed by class label, classes are sorted, and equal priors are assigned when no alternative prior dictionary is supplied. Each class receives a Gaussian log-density plus the logarithm of its prior, and the predicted label is the class with the highest resulting discriminant.

7. Detailed Methodology

7.1 Exercise 1 Simulation Pipeline

For each mean scenario, `simulate_textbook_experiment` constructs a new seeded NumPy generator, samples 100 vectors from each class distribution, concatenates the samples, and creates a ground-truth label vector. The BME and BMR predictors are then applied to all 200 observations.

The experiment driver records both sample-level information and method-level performance. For every observation it stores the scenario, sample index, true label, two feature values, and both predictions. For each method it calculates overall and class-specific error statistics. Misclassified observations receive additional diagnostic fields, including the log discriminant associated with the true class and predicted class; BMR rows also store the corresponding conditional risks.

The implementation also computes the theoretical BME error for the two-class Gaussian case

with equal covariance and equal priors. Defining

$$d_M = \sqrt{(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)}, \quad (13)$$

the Bayes error rate is

$$P_{\text{error}} = \Phi\left(-\frac{d_M}{2}\right), \quad (14)$$

where Φ is the standard normal cumulative distribution function. The function `theoretical_equal_covariance` implements this closed-form expression using `np.linalg.solve` rather than explicitly forming $\boldsymbol{\Sigma}^{-1}$.

7.2 Maximum-Likelihood Gaussian Parameter Estimation

For the multiclass data, the conditional distribution of each class is not known a priori. The implementation estimates its parameters directly from the training observations.

For a class C_c with samples $\mathbf{x}_1, \dots, \mathbf{x}_{n_c}$, the mean estimator is

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} \mathbf{x}_i. \quad (15)$$

The covariance estimator is

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{n_c} \sum_{i=1}^{n_c} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)^\top. \quad (16)$$

The divisor is explicitly n_c , which is the maximum-likelihood estimator for a Gaussian covariance parameter. This differs from the unbiased sample covariance convention that uses $n_c - 1$.

The function checks the sign returned by `np.linalg.slogdet`. A non-positive sign raises an error, so the classifier does not continue with an invalid covariance matrix.

7.3 True Leave-One-Out Evaluation

The leave-one-out procedure is implemented explicitly rather than delegated to a high-level cross-validation helper. For each observation index i , the code constructs a boolean mask containing all observations except i , fits a complete set of class Gaussian parameters on the remaining $n - 1$ samples, predicts the held-out observation, and stores its log discriminants and fold metadata.

For the Iris dataset this produces 150 fitted classifiers; for the six-dimensional Wine-equivalent dataset it produces 178 fitted classifiers. The held-out sample is never included in the parameter estimates used to predict it. This is the defining property of genuine leave-one-out evaluation and avoids direct leakage from the held-out sample into its own fold.

7.4 Train/Test Evaluation for the Normal Dataset

The two-class Normal experiment separates estimation and evaluation. Gaussian parameters are fitted from the 1000-sample training set only. The resulting class models are then applied unchanged to the independent 1000-sample test set. Performance statistics, a confusion matrix, and detailed misclassification records are written as artifacts.

8. Mathematical Formulation

The program follows a simple decision pipeline. For each class c , the classifier first estimates or receives $(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ and a prior π_c . For an input $\mathbf{x}$ it computes

$$g_c(\mathbf{x}) = -\frac{1}{2} \left[d \log(2\pi) + \log |\boldsymbol{\Sigma}_c| + (\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) \right] + \log \pi_c. \quad (17)$$

BME chooses the class maximizing $g_c(\mathbf{x})$. BMR first converts these scores into posterior probabilities and then minimizes the expected loss.

8.1 Binary Asymmetric-Risk Boundary

For the Exercise 1 loss matrix, BMR chooses class 1 whenever

$$P(C_2 \mid \mathbf{x}) < 0.5 P(C_1 \mid \mathbf{x}), \quad (18)$$

which is equivalent to

$$P(C_1 \mid \mathbf{x}) > \frac{2}{3}. \quad (19)$$

This makes the effect of the loss matrix clear: class 1 is selected only when its posterior probability exceeds $2/3$, rather than the 0.5 threshold associated with symmetric binary zero-one loss.

8.2 Error Metrics

The implementation computes the empirical overall error rate as

$$\hat{e} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i \neq \hat{y}_i], \quad (20)$$

and overall accuracy as

$$\hat{a} = 1 - \hat{e}. \quad (21)$$

For class c , the class-specific error rate is computed as the fraction of observations from class c that are predicted as something other than c . The generated CSV artifacts report these quantities as both fractions and percentages.

9. Implementation Details

9.1 Vectorization and Array Semantics

The code consistently converts inputs to NumPy arrays before numerical operations. Class-wise sample extraction is expressed with boolean masks. Concatenation of class samples uses `np.vstack`, while the corresponding labels are constructed with one-dimensional integer arrays. Thus, the classifier uses the standard tabular representation in which each row is an observation and each column is a feature.

9.2 Positive-Definiteness Checks

A multivariate Gaussian density requires a valid covariance matrix. The implementation uses `np.linalg.slogdet` as a practical validity check. A non-positive determinant sign triggers a

ValueError. Because a symmetric positive-definite matrix has a positive determinant, this detects the main invalid case relevant to the Gaussian density formula.

This check is useful, but it does not assess numerical conditioning in full. A matrix can have a positive determinant and still be poorly conditioned. The code does not explicitly test the condition number or apply covariance regularization.

9.3 Artifact Naming and Data Lineage

The output directory receives logically named CSV files for raw generated samples, model performance summaries, confusion matrices, first-fold parameter estimates, misclassification details, and a compact overall summary. The final JSON manifest records the main configuration and dataset definitions. This provides a more auditable record than a workflow that only prints values to standard output.

9.4 Source-Level Traceability

The following line ranges identify the corresponding parts of the supplied implementation.

Component	Lines	What the source establishes
Project/source note	9–20	Assignment context, source discrepancy, and missing-data substitutions.
Configuration	23–44	Imports, seed, output directory, and runtime meta-data.
Gaussian primitive	74–86	Log-density implementation and covariance validation.
Posterior conversion	90–96	Numerically stabilized posterior normalization.
Binary BME/BMR	98–120	Minimum-error and minimum-risk decisions.
Error diagnostics	122–149	Confusion matrix and error/accuracy metrics.
Exercise 1 simulation	151–176	Gaussian sampling and theoretical equal-covariance error.
Exercise 1 artifact generation	177–267	Tables, sample records, and misclassification records.
Plotting helpers	269–334	Optional visualization outputs present in the source but omitted from this report.
ML parameter estimation	348–368	Class mean and covariance estimation.
Multiclass BME	371–386	Generic multiclass Gaussian decision rule.
Leave-one-out	388–415	Refit-and-predict loop for every held-out observation.
Iris workflow	417–521	Feature ordering, LOO classification, metrics, parameters, diagnostics.
Liquid/Wine-equivalent	523–640	Six-feature substitution and 178-fold LOO workflow.
Normal workflow	642–739	Seeded data reconstruction and train/test BME evaluation.
Normal plotting	741–789	Optional two-dimensional visualization.
Manifest	791–886	Summary table and reproducibility metadata.

10. Execution Workflow

With the same source and software environment, the execution is deterministic with respect to its random-number generation. The program first creates the output directory and records the execution environment. It then executes the two-class Gaussian experiment, saves its tabular artifacts, and produces optional plots. Next, it loads Iris and performs 150 leave-one-out fits. It then loads Wine as the documented Liquid-equivalent substitute and performs 178 leave-one-out fits. Finally, it reconstructs the Normal training and testing sets, fits two Gaussian class models on the training data, evaluates the classifier on the test data, and writes the associated artifacts.

The final block builds a compact cross-experiment performance table and writes a JSON manifest. Because the earlier blocks run before the manifest is written, the manifest records the configuration used to produce the generated files.

11. Design Decisions and Rationale

11.1 Why Log Probabilities Are Used

Direct Gaussian densities can become extremely small as dimensionality grows or as observations move into the tails. Storing such values directly increases the risk of underflow. Log probabilities transform products into sums and permit a stable maximum comparison. The implementation therefore represents the core discriminants in log space.

11.2 Why Maximum-Likelihood Covariance Uses n

The assignment specifies maximum-likelihood Gaussian parameter estimation. The code follows that requirement directly by dividing the centered outer-product sum by the class sample count. No unbiased $n - 1$ correction is introduced.

11.3 Why Leave-One-Out Is Implemented Manually

The assignment requires one classifier per omitted observation. The explicit loop makes that protocol visible and auditable: each fold reconstructs the training set, estimates class parameters, predicts one sample, and stores fold metadata. A library abstraction could shorten the source, but the explicit loop matches the assignment requirement and makes the evaluation procedure easy to inspect.

11.4 Why the Missing Data Are Reconstructed or Substituted

The source explicitly documents that the supplied environment lacks the original Normal files and Liquid database. Reconstructing the Normal files from stated distribution parameters preserves the intended probabilistic structure. For the Liquid case, the code selects Wine because its documented size, class counts, and chemical-measurement character align with the assignment description, while explicitly retaining only six original features. This is a practical reproducibility decision, but it also means that the resulting outputs are not exact reproductions of the experiment based on the unavailable original data.

12. Computational Considerations

Let K denote the number of classes and d the feature dimension. For a single sample, one Gaussian log-density evaluation requires a covariance inversion, a log-determinant calculation,

and a quadratic form. A direct dense matrix inverse is nominally $O(d^3)$, while the quadratic form is $O(d^2)$. Because the supplied low-level routine computes these quantities every time it is called, repeated classification with an unchanged covariance performs repeated matrix factorizations that could instead be cached.

For a dataset of N observations, the multiclass classifier therefore has a repeated numerical cost on the order of $O(NKd^3)$ under the current implementation strategy, ignoring lower-order operations. With cached inverse covariances and log-determinants, the per-prediction cost could be reduced substantially after one-time parameter preprocessing.

The leave-one-out driver adds another multiplicative factor because parameters are refit N times. If covariance estimation is performed naively from each training fold, the complete LOO procedure is inherently much more expensive than a single train/test fit. The current implementation favors directness and traceability over aggressive reuse of intermediate computations.

Memory usage is modest for the assignment-scale datasets. Generated samples, fold records, and diagnostics fit naturally in in-memory NumPy arrays and pandas DataFrames. The main artifact cost comes from storing multiple CSV representations of the same underlying information rather than from the numerical arrays themselves.

13. Reproducibility and Reliability

The implementation includes several concrete reproducibility mechanisms. A fixed module-level random seed is combined with deterministic offsets for independent scenarios and Normal training/testing files. The environment records Python, NumPy, and pandas versions. The output directory is created programmatically. Important model and dataset configuration is serialized into a JSON manifest.

The experiment code also makes its data lineage explicit. In particular, the Wine substitution is labeled in the generated filenames and in the manifest, which reduces the risk that a future reader mistakes the substitute dataset for the missing Liquid file.

Reproducibility still depends on the surrounding software environment. Numerical libraries, linear-algebra backends, and floating-point details can produce tiny differences even when random seeds are fixed. The program does not pin package versions or construct a formal environment lockfile. The manifest records versions but does not install or enforce them.

14. Limitations and Technical Considerations

14.1 Missing Assignment Data

The most important external limitation is the absence of the original Normal and Liquid inputs. Exercise 2b is explicitly a substitute experiment, and Exercise 2c reconstructs synthetic files from stated parameters. Any numerical result produced by those blocks should therefore be interpreted as an execution of the documented substitute or reconstruction, not as a byte-for-byte reproduction of missing source data.

14.2 Repeated Matrix Inversion

The low-level Gaussian routine calls `np.linalg.inv` and `np.linalg.slogdet` each time it is invoked. For a covariance matrix that is reused for thousands of samples, caching its inverse

and log-determinant would be more efficient. A Cholesky factorization would also provide a numerically attractive route for positive-definite covariance matrices.

14.3 Conditioning and Covariance Regularization

The implementation rejects non-positive-definite covariance matrices, but it does not regularize nearly singular covariances. In higher-dimensional or small-sample regimes, this could make the classifier sensitive to conditioning. Such a limitation is methodological only insofar as the Gaussian covariance model itself is retained without shrinkage or diagonal loading; it is not a defect in the basic Bayes formula.

14.4 Equal Priors

Equal priors are hard-coded or used by default in the assignment workflows. This is faithful to the stated assignment setup, but it limits applicability to settings in which empirical class frequencies or domain knowledge imply unequal prior probabilities.

14.5 Plotting Code and Reporting Scope

The source includes plotting functions and saves images. Those routines are computationally independent from the core classifier and do not alter the main decision logic. This report intentionally excludes those visual outputs because the requested deliverable is a code and methodology analysis without figures.

15. Overall Technical Assessment

The implementation is technically coherent for its intended educational use. Its strongest features are the explicit separation of Bayesian decision primitives from experiment drivers, the consistent use of logarithmic Gaussian scores, the correct maximum-likelihood covariance convention, and the direct implementation of true leave-one-out refitting. The code also shows awareness of reproducibility through fixed random generation, explicit artifact naming, environment logging, and a final configuration manifest.

The project also illustrates several research-oriented software practices. It turns textbook probability models into executable numerical procedures, preserves the distinction between likelihoods, posteriors, and decision risks, and stores intermediate information sufficiently richly to support later audit or analysis. The source comments also document discrepancies between the assignment text and the available data rather than leaving them implicit.

The main engineering opportunities are numerical optimization and stronger environment encapsulation. Caching Gaussian parameters, avoiding repeated matrix inversions, and formalizing dependencies would make the implementation more scalable and easier to reproduce across machines. The other major caveat is data provenance: the missing-file substitutions must remain clearly labeled in any downstream analysis.

16. Conclusion

The supplied program implements a complete Bayesian Gaussian-classification workflow for a statistical pattern-recognition assignment. Its mathematical core is a multivariate Gaussian

generative model evaluated in log space, followed by either a minimum-error decision or expected-loss minimization under a specified loss matrix. The same building blocks are extended to maximum-likelihood multiclass Gaussian models and evaluated through explicit leave-one-out or train/test procedures.

The implementation is useful as a technical example because it does more than produce predictions. It exposes the intermediate decision statistics, stores fold and misclassification information, records the model configuration, and writes structured artifacts that make later inspection possible. At the same time, the analysis identifies the important boundaries of what the source establishes: the missing original datasets prevent exact reproduction of those portions of the assignment, and the current numerical implementation favors transparency over computational caching.

Overall, the code is a sound implementation of classical statistical pattern-recognition concepts, presented in a form that is understandable, auditable, and reproducible at the scale of the assignment.